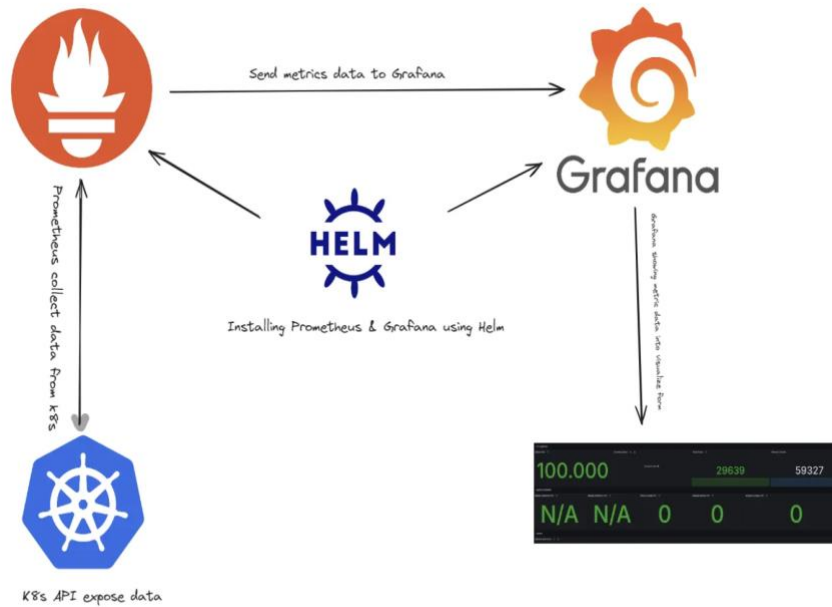


Kubernetes Monitoring with Prometheus & Grafana



Developed by: Sagar Pyakurel

1. Project Overview

This project implements Kubernetes cluster monitoring using:

- **Kubernetes** – Container orchestration platform
- **Helm** – Package manager for Kubernetes
- **Prometheus** – Metrics collection and monitoring
- **Grafana** – Metrics visualization and dashboards
- **kube-state-metrics** – Kubernetes object/state metrics
- **Node Exporter** – Node-level hardware/system metrics
- **Alertmanager** – Alert management

The monitoring stack was deployed using the **kube-prometheus-stack Helm chart**.

2. Kubernetes Environment

The Kubernetes cluster contains four nodes:

```
kubectl get nodes
```

Output:

NAME	STATUS	ROLES	AGE	VERSION
my-cluster	Ready	control-plane	...	v1.35.1
my-cluster-m02	Ready	<none>	...	v1.35.1
my-cluster-m03	Ready	<none>	...	v1.35.1
my-cluster-m04	Ready	<none>	...	v1.35.1

What the command does

`kubectl get nodes` displays all nodes in the Kubernetes cluster and shows their:

- Name, Status, Role, Age, Kubernetes version

All four nodes were in the **Ready** state.

3. Monitoring Namespace

The monitoring components were deployed in a dedicated namespace called `monitoring`.

```
kubens
```

This displays the available Kubernetes namespaces.

Then:

```
kubens monitoring
```

What it does

`kubens monitoring` changes the **active namespace** for the current `kubectl` context to `monitoring`.

After this, commands such as:

```
kubectl get pods
```

automatically operate against the `monitoring` namespace.

4. Add Prometheus Community Helm Repository

The Prometheus Community Helm repository was added using:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

What it does

This registers the Prometheus Community Helm chart repository with Helm.

The repository contains charts such as:

```
kube-prometheus-stack  
prometheus  
alertmanager
```

Verify the repository:

```
helm repo list
```

The configured repositories included:

```
prometheus-community  
bitnami
```

5. Update Helm Repositories

```
helm repo update
```

What it does

Downloads the latest chart information from the configured Helm repositories.

This ensures Helm has the latest available chart versions before installation.

6. Install kube-prometheus-stack

The monitoring stack was installed with:

```
helm install prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set prometheus.service.nodePort=30000 \
  --set grafana.service.nodePort=31000 \
  --set grafana.service.type=NodePort \
  --set prometheus.service.type=NodePort
```

What this command does

This installs the kube-prometheus-stack chart into the monitoring namespace.

The Helm release name is:

prometheus-stack

The chart automatically creates and configures several monitoring components.

Important options

Option	Purpose
prometheus-stack	Helm release name
prometheus-community/kube-prometheus-stack	Helm chart being installed
--namespace monitoring	Installs everything in monitoring
prometheus.service.nodePort=30000	Exposes Prometheus through NodePort 30000
grafana.service.nodePort=31000	Exposes Grafana through NodePort 31000
grafana.service.type=NodePort	Changes Grafana service to NodePort
prometheus.service.type=NodePort	Changes Prometheus service to NodePort

7. Verify Monitoring Pods

```
kubectl get pods -n monitoring
```

The installation created the following major components:

alertmanager
prometheus
grafana
kube-prometheus-operator
kube-state-metrics
node-exporter

What each component does

Prometheus

Collects and stores monitoring metrics from the Kubernetes environment.

Grafana

Uses Prometheus metrics to create dashboards and visualizations.

Alertmanager

Handles alerts generated by Prometheus.

Prometheus Operator

Manages Prometheus-related Kubernetes resources and configuration.

kube-state-metrics

Exposes metrics about Kubernetes objects such as:

- Pods
- Deployments
- Nodes
- Services
- ReplicaSets

Node Exporter

Collects operating-system and node-level metrics such as:

- CPU
 - Memory
 - Disk
 - Network
-

8. Check Monitoring Services

```
kubectl get svc
```

Because monitoring was made the active namespace with kubens, this command displayed services from the monitoring namespace.

Important services included:

```
prometheus-stack-grafana
prometheus-stack-kube-prom-prometheus
prometheus-stack-kube-prom-alertmanager
prometheus-stack-kube-state-metrics
prometheus-stack-prometheus-node-exporter
```

The important exposed services were:

Grafana:
80:31000

Prometheus:
9090:30000

Therefore:

Grafana NodePort → 31000
Prometheus NodePort → 30000

9. Port Forward Prometheus

You also used port forwarding:

```
kubectl port-forward svc/prometheus-stack-kube-prom-prometheus \
  9090:9090 \
  -n monitoring \
  --address=0.0.0.0
```

What it does

It forwards:

Local port 9090

↓

Prometheus service port 9090

--address=0.0.0.0 makes the port-forward listen on all network interfaces rather than only localhost.

You then ran it in the background:

```
kubectl port-forward svc/prometheus-stack-kube-prom-prometheus \
  9090:9090 \
  -n monitoring \
  --address=0.0.0.0 &
```

The & sends the process to the background so the terminal can be used for other commands.

10. Port Forward Grafana

You used:

```
kubectl port-forward svc/prometheus-stack-grafana \
  3000:80 \
  --address=0.0.0.0
```

What it does

It forwards:

Local port 3000

↓

Grafana service port 80

You also ran it in the background:

```
kubectl port-forward svc/prometheus-stack-grafana \
  3000:80 \
  --address=0.0.0.0 &
```

This allowed Grafana to remain accessible while you continued using the terminal.

11. Get Grafana Admin Password

The Grafana password was stored in a Kubernetes Secret.

You retrieved it with:

```
kubectl get secret prometheus-stack-grafana \
  -n monitoring \
  -o jsonpath="{.data.admin-password}"
```

The result was Base64 encoded.

You then decoded it with:

```
echo 'BASE64_PASSWORD' | base64 --decode
```

What it does

The command:

```
kubectl get secret
```

retrieves the Kubernetes Secret.

The option:

```
-o jsonpath="{.data.admin-password}"
```

extracts only the admin-password field.

Then:

```
base64 --decode
```

converts the encoded password into its actual value.

12. Verify All Kubernetes Resources

You checked all namespaces with:

```
kubectl get pods -A
```

What it does

The -A option means:

```
--all-namespaces
```

It displays pods from every namespace.

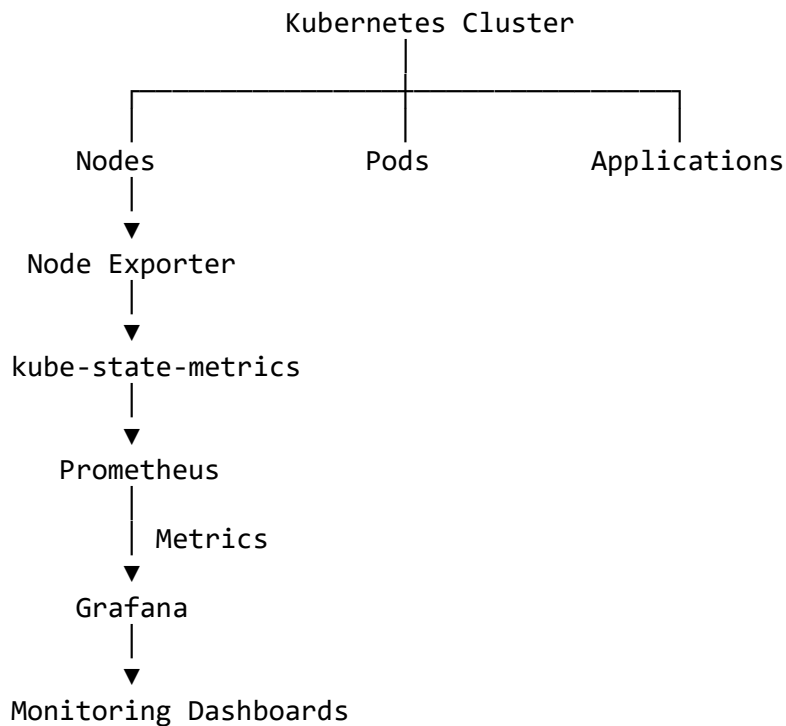
This confirmed that the cluster contained:

```
chat-app  
ingress-nginx  
kube-system  
monitoring
```

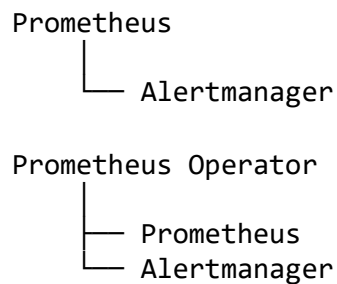
and that the monitoring components were running alongside the application workloads.

13. Final Monitoring Architecture

The resulting architecture is:



The installed stack also includes:



14. Useful Verification Commands

Check monitoring pods:

```
kubectl get pods -n monitoring
```

Check monitoring services:

```
kubectl get svc -n monitoring
```

Check nodes:

```
kubectl get nodes
```

Check everything across namespaces:

```
kubectl get pods -A
```

Check Helm releases:

```
helm list -n monitoring
```

Expected Helm release:

```
prometheus-stack
```

15. Final Result

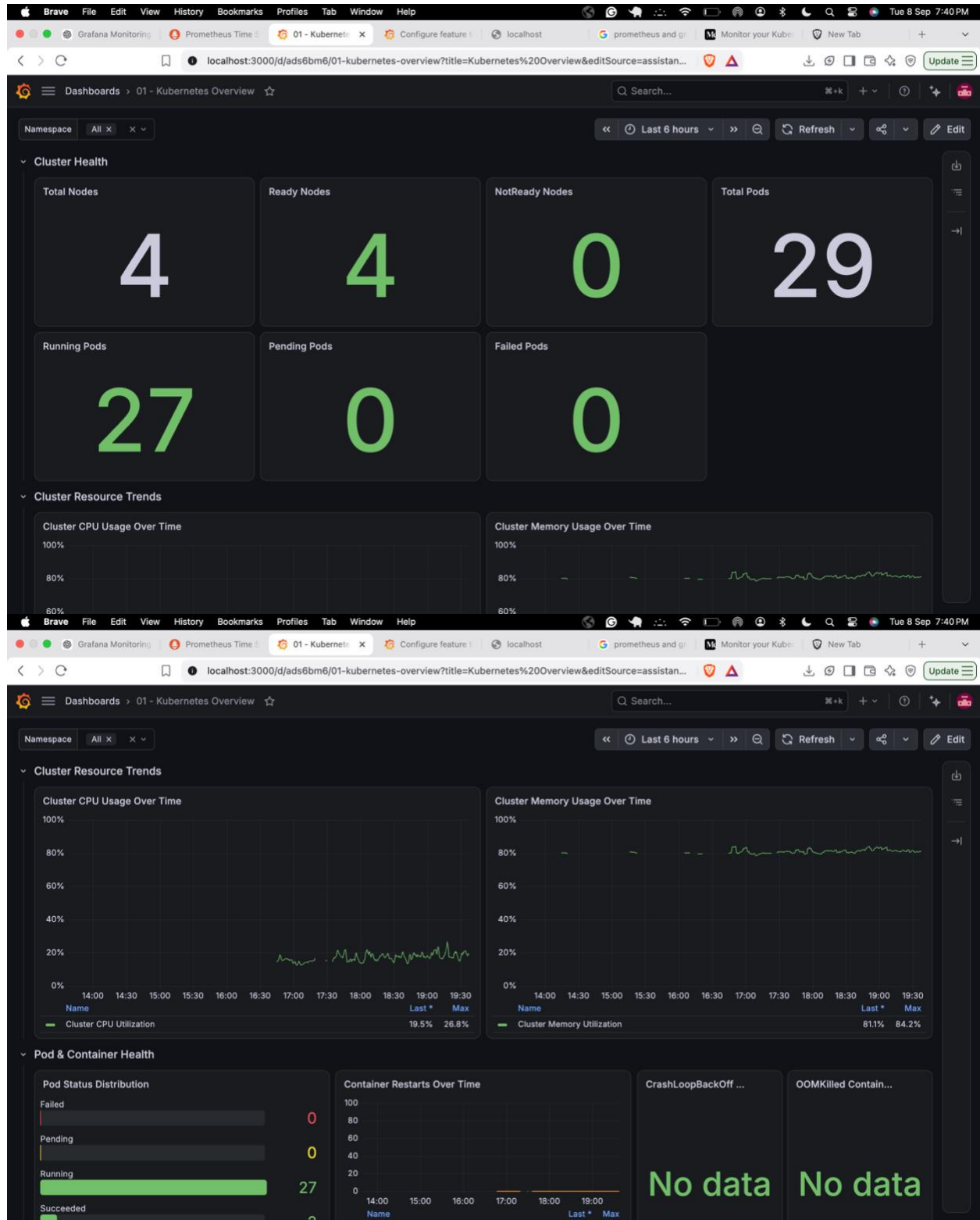
The project successfully deployed a complete Kubernetes monitoring stack using Helm.

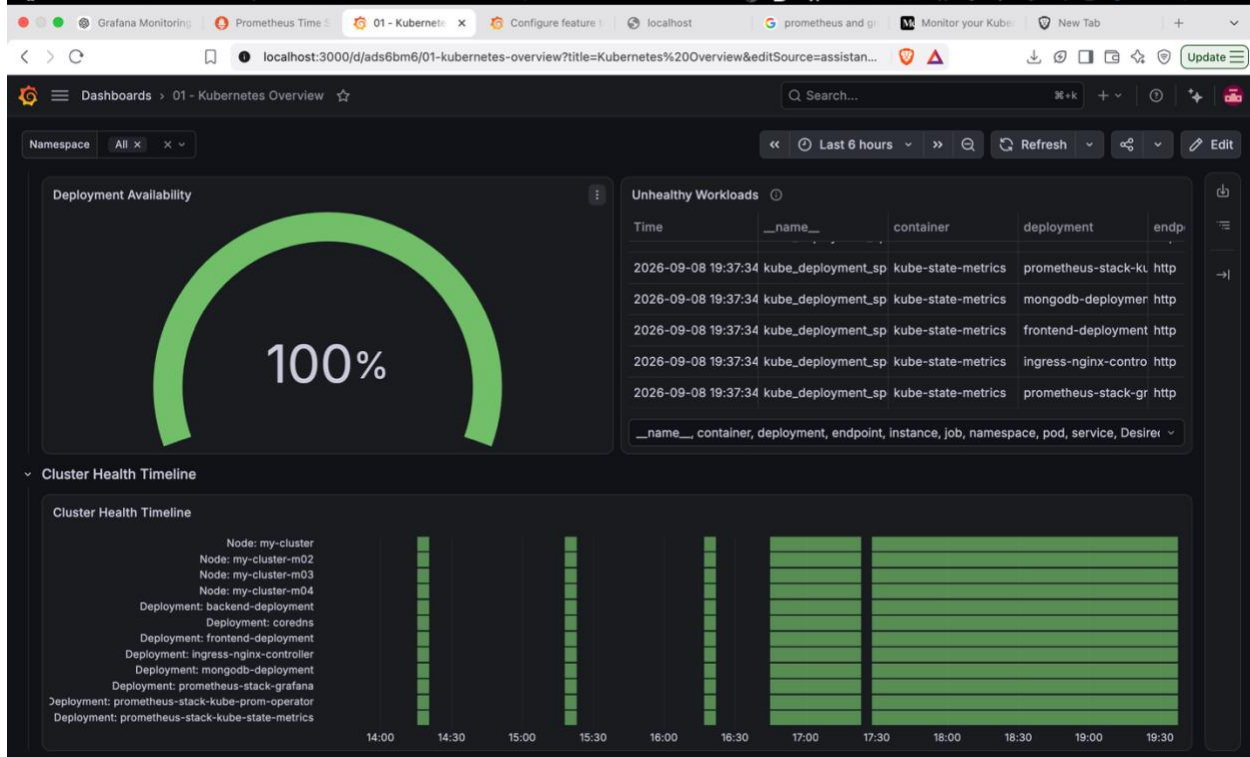
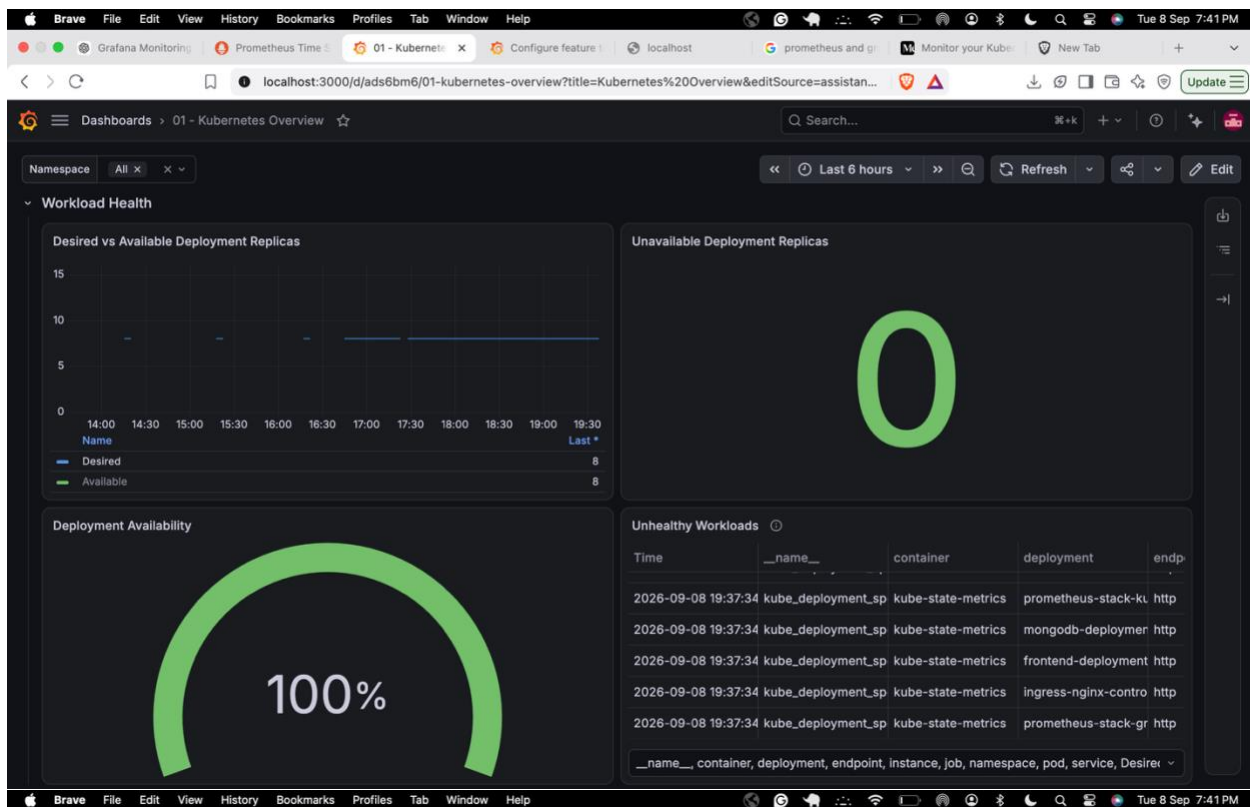
Implemented

- Kubernetes cluster monitoring
- Prometheus metrics collection
- Grafana visualization
- Node-level monitoring with Node Exporter
- Kubernetes object monitoring with kube-state-metrics
- Alertmanager
- Prometheus Operator
- NodePort access for Prometheus and Grafana
- Port-forward access for Prometheus and Grafana
- Grafana monitoring dashboards

The monitoring stack provides visibility into the Kubernetes cluster, nodes, pods, workloads, CPU/memory usage, networking, and application health.

16. Images of Grafana Dashboard





```
Terminal Shell Edit View Window Help
sagarpyakurel : monitoring # kubectl get pods
NAME                                     READY   STATUS    RESTARTS   AGE
alertmanager-prometheus-stack-kube-prom-alertmanager-0 2/2     Running   0           24h
prometheus-prometheus-stack-kube-prom-prometheus-0      2/2     Running   0           24h
prometheus-stack-grafana-59cf6586d8-qscgt               3/3     Running   0           24h
prometheus-stack-kube-prom-operator-76f944bff-dz847      1/1     Running   0           24h
prometheus-stack-kube-state-metrics-584998dd9-hgm5z      1/1     Running   0           24h
prometheus-stack-prometheus-node-exporter-7r9pk         1/1     Running   0           24h
prometheus-stack-prometheus-node-exporter-8rxs9         1/1     Running   0           24h
prometheus-stack-prometheus-node-exporter-rsqbx         1/1     Running   0           24h
prometheus-stack-prometheus-node-exporter-tcb84         1/1     Running   0           24h

sagarpyakurel : monitoring # kubectl get svc
NAME                                     TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)                                     AGE
alertmanager-operated                  ClusterIP      None             <none>            9093/TCP,9094/TCP,9094/UDP                24h
prometheus-operated                    ClusterIP      None             <none>            9090/TCP                                   24h
prometheus-stack-grafana                NodePort       10.109.148.3    <none>            80:31000/TCP                              24h
prometheus-stack-kube-prom-alertmanager ClusterIP       10.109.254.6    <none>            9093/TCP,8080/TCP                         24h
prometheus-stack-kube-prom-operator      ClusterIP       10.100.1.131    <none>            443/TCP                                    24h
prometheus-stack-kube-prom-prometheus    NodePort       10.98.248.173   <none>            9090:30000/TCP,8080:31394/TCP             24h
prometheus-stack-kube-state-metrics      ClusterIP       10.105.124.167  <none>            8080/TCP                                   24h
prometheus-stack-prometheus-node-exporter ClusterIP       10.103.18.200   <none>            9100/TCP                                    24h

sagarpyakurel : monitoring # |
```

16. Extra Work: Alert system setup contact point

I also added Grafana Contact Points and researched the major Kubernetes alert rules needed for the monitoring setup. I created and configured the major alert rules for critical conditions such as Node NotReady, CrashLoopBackOff, Deployment Unavailable, OOMKilled, high CPU/memory usage, critical disk usage, and HPA scaling issues. The alert rules were also tested and verified in Grafana to ensure they work correctly with the Prometheus metrics and notification setup.

created secret from terminal:

```
kubectl create secret generic grafana-smtp \
  -n monitoring \
  --from-literal=GF_SMTP_USER='mygmail@gmail.com' \
  --from-literal=GF_SMTP_PASSWORD='my_gmail_password'
```

monitoring-values.yaml

```
grafana:
  grafana.ini:
    smtp:
```

```
enabled: true

host: smtp.gmail.com:587

user: $__env{GF_SMTP_USER}

password: $__env{GF_SMTP_PASSWORD}

from_address: yourgmail@gmail.com

from_name: Grafana

startTLS_policy: MandatoryStartTLS
```

env:

```
GF_SMTP_USER:

  valueFrom:

    secretKeyRef:

      name: grafana-smtp

      key: user

GF_SMTP_PASSWORD:

  valueFrom:

    secretKeyRef:

      name: grafana-smtp

      key: password
```

Upgrade Helm

```
helm upgrade prometheus-stack prometheus-community/kube-prometheus-stack \

-n monitoring \

-f monitoring-values.yaml
```